



INGENIERIA EN SISTEMAS
PROGRAMACION III
QUINTO CICLO
ING. MBA DAVID ALVAREZ

Tarea XI - Tablas Hash

EMILY HERRERA 9941-24-1489

CODIGO COMPLETO JAVA REALIZADO EN INTELLIJ IDEA

CLASS ESTUDIANTE

Requisito	Dónde se ve
Leer CSV	cargarCSV()
HashMap	HashMap<Integer, Estudiante>
Clave	student_id
Duplicados	containsKey()
Buscar	menú opción 2
Eliminar	menú opción 3
Total	estudiantes.size()
Evidencia HashMap	put(), get(), remove()

VENTAJA

En C++ fue necesario implementar manualmente la tabla hash utilizando arreglos, nodos y listas enlazadas para resolver colisiones.

En Java, HashMap ya proporciona una implementación optimizada y abstraída, permitiendo insertar, buscar y eliminar elementos de forma más sencilla.

```
package edu.umg;

public class Estudiante {

    int studentId;
    String fullName;
    String career;
    int semester;
    double gpa;
    int skillScore;

    public Estudiante(int studentId,
                      String fullName,
                      String career,
                      int semester,
                      double gpa,
```

```

        int skillScore) {

    this.studentId = studentId;
    this.fullName = fullName;
    this.career = career;
    this.semester = semester;
    this.gpa = gpa;
    this.skillScore = skillScore;
}

@Override
public String toString() {

    return "ID: " + studentId +
           "\nNombre: " + fullName +
           "\nCarrera: " + career +
           "\nSemestre: " + semester +
           "\nGPA: " + gpa +
           "\nSkill Score: " + skillScore;
}
}

```

CLASS MAIN

```

package edu.umg;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.HashMap;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        // HASHMAP
        HashMap<Integer, Estudiante> estudiantes = new HashMap<>();

        // SCANNER
        Scanner scanner = new Scanner(System.in);

        System.out.println("=====");
        System.out.println("Nombre: Emily Monterroso");
        System.out.println("Carné: 9941-24-1489");
        System.out.println("=====");

        // CARGAR CSV
        cargarCSV(estudiantes);

        int opcion;

        do {

            System.out.println("\n===== MENU =====");
            System.out.println("1. Mostrar estudiantes");
            System.out.println("2. Buscar estudiante");
            System.out.println("3. Eliminar estudiante");
            System.out.println("4. Mostrar estadísticas");
            System.out.println("5. Salir");
            System.out.print("Seleccione una opcion: ");

```

```

opcion = scanner.nextInt();

switch (opcion) {

    case 1:

        mostrarEstudiantes(estudiantes);
        break;

    case 2:

        buscarEstudiante(estudiantes, scanner);
        break;

    case 3:

        eliminarEstudiante(estudiantes, scanner);
        break;

    case 4:

        mostrarEstadisticas(estudiantes);
        break;

    case 5:

        System.out.println("Saliendo del programa...");
        break;

    default:

        System.out.println("Opcion invalida.");
}

} while (opcion != 5);

scanner.close();
}

public static void cargarCSV(HashMap<Integer, Estudiante> estudiantes)
{

    String archivo = "estudiantes.csv";

    try {

        BufferedReader br = new BufferedReader(
            new FileReader(archivo)
        );

        String linea;

        br.readLine();

        while ((linea = br.readLine()) != null) {

            String[] datos = linea.split(",");

            int id = Integer.parseInt(datos[0]);
            String nombre = datos[1];

```

```

        String carrera = datos[2];
        int semestre = Integer.parseInt(datos[3]);
        double gpa = Double.parseDouble(datos[4]);
        int skillScore = Integer.parseInt(datos[5]);

        if (!estudiantes.containsKey(id)) {

            Estudiante estudiante = new Estudiante(
                id,
                nombre,
                carrera,
                semestre,
                gpa,
                skillScore
            );

            estudiantes.put(id, estudiante);

        } else {

            System.out.println(
                "ID duplicado encontrado: " + id
            );

        }

    }

    br.close();

    System.out.println("\nCSV cargado correctamente.");

} catch (IOException e) {

    System.out.println("Error al leer el archivo CSV.");
    System.out.println(e.getMessage());

}

}

public static void mostrarEstudiantes(
    HashMap<Integer, Estudiante> estudiantes) {

    System.out.println("\n=== ESTUDIANTES ===");

    for (Integer key : estudiantes.keySet()) {

        System.out.println("-----");
        System.out.println(estudiantes.get(key));

    }

}

public static void buscarEstudiante(
    HashMap<Integer, Estudiante> estudiantes,
    Scanner scanner) {

    System.out.print("Ingrese ID a buscar: ");

    int id = scanner.nextInt();

    if (estudiantes.containsKey(id)) {

```

```

        System.out.println("\n=== ESTUDIANTE ENCONTRADO ===");
        System.out.println(estudiantes.get(id));

    } else {

        System.out.println("ESTUDIANTE NO ENCONTRADO.");
    }
}

public static void eliminarEstudiante(
    HashMap<Integer, Estudiante> estudiantes,
    Scanner scanner) {

    System.out.print("Ingrese ID a eliminar: ");

    int id = scanner.nextInt();

    if (estudiantes.containsKey(id)) {

        estudiantes.remove(id);

        System.out.println("Estudiante eliminado.");
    } else {

        System.out.println("No existe estudiante.");
    }
}

public static void mostrarEstadisticas(
    HashMap<Integer, Estudiante> estudiantes) {

    System.out.println("\n=== ESTADISTICAS ===");

    System.out.println(
        "Total estudiantes: " + estudiantes.size()
    );

    System.out.println(
        "Estructura utilizada: HashMap<Integer, Estudiante>"
    );
}
}

```

POM

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>edu.umg</groupId>
    <artifactId>hash</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>

```

```
<maven.compiler.source>25</maven.compiler.source>
<maven.compiler.target>25</maven.compiler.target>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

</project>
```

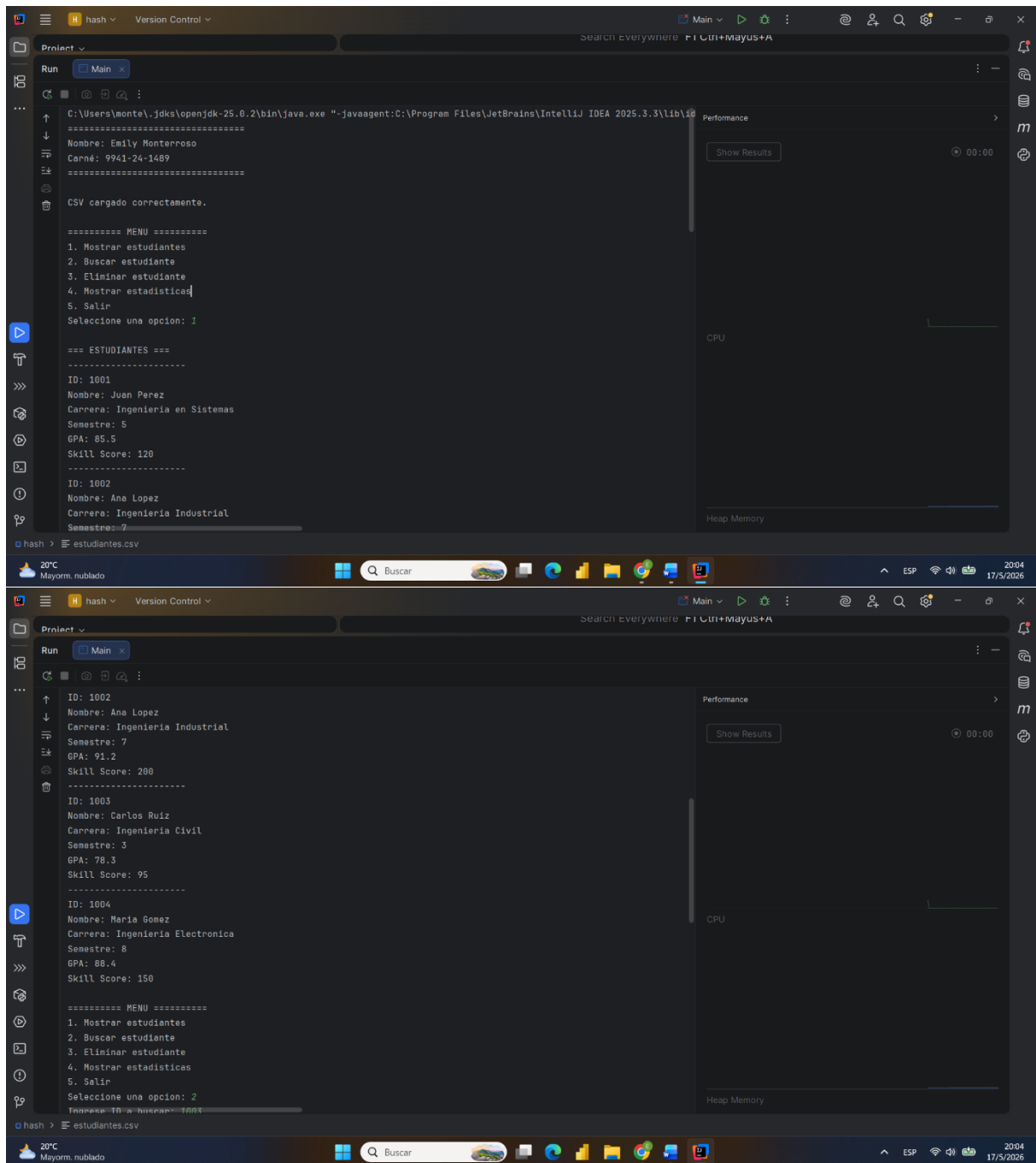
ESTUDIANTES CSV

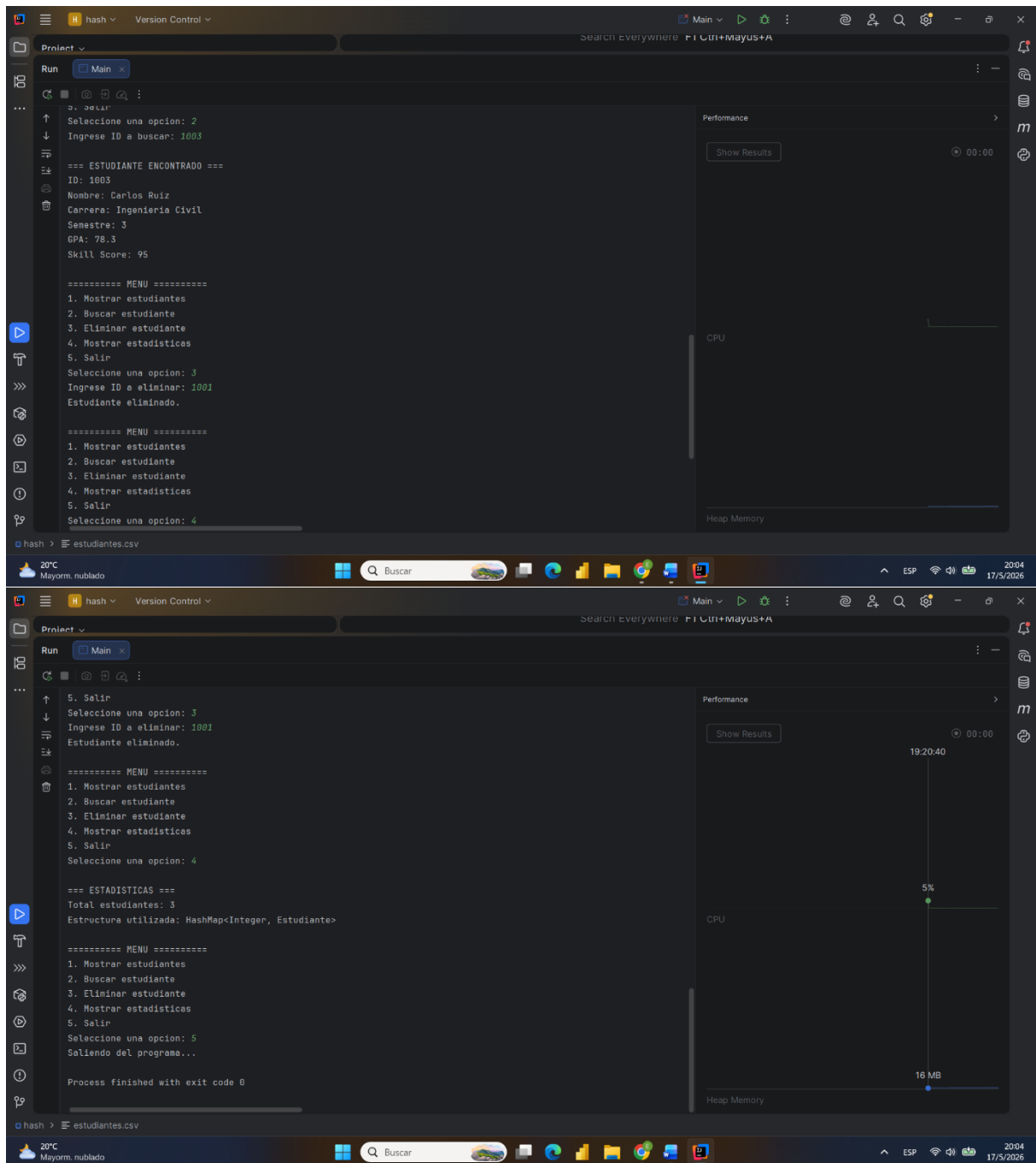
```
student_id,full_name,career,semester,gpa,skill_score
1001,Juan Perez,Ingenieria en Sistemas,5,85.50,120
1002,Ana Lopez,Ingenieria Industrial,7,91.20,200
1003,Carlos Ruiz,Ingenieria Civil,3,78.30,95
1004,Maria Gomez,Ingenieria Electronica,8,88.40,150
```

INTERPRETACION

The screenshot shows a GitHub repository named 'tarea-hash-csv' by user 'EMILY-UMG'. The file 'estudiantes.csv' is selected, showing its content in a table format. The table has 5 columns: student_id, full_name, career, semester, gpa, and skill_score. The data rows are as follows:

	student_id	full_name	career	semester	gpa	skill_score
1	1001	Juan Perez	Ingenieria en Sistemas	5	85.50	120
2	1002	Ana Lopez	Ingenieria Industrial	7	91.20	200
3	1003	Carlos Ruiz	Ingenieria Civil	3	78.30	95
4	1004	Maria Gomez	Ingenieria Electronica	8	88.40	150





CODIGO COMPLETO C++ REALIZADO EN CLION

- ✓ El sistema carga automáticamente los estudiantes desde un archivo CSV utilizando la función cargarCSV().
hash.cargarCSV("../estudiantes.csv");
- ✓ Cada estudiante leído desde el archivo CSV es insertado en la tabla hash mediante la función insertar().
Bucket [1]:
(1031, Maria Gomez) ->
(1021, Carlos Ruiz) ->
- ✓ Se utilizó student_id como clave principal para calcular el bucket correspondiente en la tabla hash.
funcionHash(estudiante.student_id)
- ✓ La tabla hash fue implementada manualmente utilizando arreglos y nodos dinámicos, sin utilizar estructuras STL.
class HashTable
- ✓ Las colisiones fueron resueltas mediante Separate Chaining utilizando listas enlazadas.
Bucket [1]:
(1031, Maria Gomez) ->
(1021, Carlos Ruiz) ->
(1011, Ana Lopez) ->
(1001, Juan Perez) ->
FINAL
- ✓ Esto puede observarse en el Bucket [1]. Antes de insertar un estudiante, el sistema verifica si el student_id ya existe en la tabla hash.
if (buscar(estudiante.student_id) != nullptr)
- ✓ El sistema permite buscar estudiantes utilizando su student_id mediante la función buscar().
2. Buscar estudiante
- ✓ El sistema permite eliminar estudiantes de la tabla hash utilizando su student_id.
3. Eliminar estudiante
- ✓ El programa muestra el total de estudiantes almacenados en la tabla hash.
TOTAL ESTUDIANTES: 4

- ✓ El sistema contabiliza las colisiones generadas durante la inserción de estudiantes.

TOTAL COLISIONES: 3

- ✓ El factor de carga se calcula dividiendo la cantidad de estudiantes entre el tamaño total de buckets.

FACTOR CARGA: 0.4

- ✓ El sistema muestra el contenido completo de cada bucket de la tabla hash, incluyendo las listas enlazadas.

Bucket [0]: FINAL

Bucket [1]: ...

Bucket [2]: FINAL

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>

using namespace std;

struct Estudiante {
    int student_id;
    string full_name;
    string career;
    int semester;
    float gpa;
    int skill_score;
};

struct Nodo {
    Estudiante estudiante;
    Nodo* siguiente;
};

class HashTable {
private:
    static const int SIZE = 10;

    Nodo* tabla[SIZE];

    int totalElementos;
    int totalColisiones;

    int funcionHash(int id) {
        return id % SIZE;
    }

public:
```

```

HashTable() {

    for (int i = 0; i < SIZE; i++) {

        tabla[i] = nullptr;

    }

    totalElementos = 0;
    totalColisiones = 0;

}

bool insertar(Estudiante estudiante) {

    int indice = funcionHash(estudiante.student_id);

    if (buscar(estudiante.student_id) != nullptr) {

        cout << "ID duplicado.\n";
        return false;

    }

    Nodo* nuevo = new Nodo;

    nuevo->estudiante = estudiante;

    nuevo->siguiente = tabla[indice];

    if (tabla[indice] != nullptr) {

        totalColisiones++;

    }

    tabla[indice] = nuevo;

    totalElementos++;

    return true;

}

Estudiante* buscar(int id) {

    int indice = funcionHash(id);

    Nodo* actual = tabla[indice];

    while (actual != nullptr) {

        if (actual->estudiante.student_id == id) {

            return &actual->estudiante;

        }

        actual = actual->siguiente;

    }

    return nullptr;

}

bool eliminar(int id) {

    int indice = funcionHash(id);

```

```

    Nodo* actual = tabla[indice];

    Nodo* anterior = nullptr;

    while (actual != nullptr) {

        if (actual->estudiante.student_id == id) {

            if (anterior == nullptr) {

                tabla[indice] = actual->siguiente;

            } else {

                anterior->siguiente = actual->siguiente;

            }

            delete actual;

            totalElementos--;

            return true;

        }

        anterior = actual;
        actual = actual->siguiente;

    }

    return false;
}

void mostrarTabla() {

    cout << "\n=== TABLA HASH ===\n";

    for (int i = 0; i < SIZE; i++) {

        cout << "Bucket [" << i << "]: ";

        Nodo* actual = tabla[i];

        while (actual != nullptr) {

            cout << "("
                << actual->estudiante.student_id
                << ", "
                << actual->estudiante.full_name
                << ") -> ";

            actual = actual->siguiente;

        }

        cout << "FINAL\n";

    }

}

void mostrarEstadisticas() {

    cout << "\n=== ESTADISTICAS ===\n";

```

```

        cout << "TOTAL ESTUDIANTES: "
              << totalElementos << endl;

        cout << "TOTAL COLISIONES: "
              << totalColisiones << endl;

        cout << "FACTOR CARGA: "
              << (float) totalElementos / SIZE << endl;
    }

    void cargarCSV(string nombreArchivo) {

        ifstream archivo(nombreArchivo);

        if (!archivo.is_open()) {

            cout << "NO SE PUEDE ABRIR CSV.\n";
            return;
        }

        string linea;

        getline(archivo, linea);

        while (getline(archivo, linea)) {

            stringstream ss(linea);

            string idStr;
            string nombre;
            string carrera;
            string semestreStr;
            string gpaStr;
            string skillStr;

            getline(ss, idStr, ',');
            getline(ss, nombre, ',');
            getline(ss, carrera, ',');
            getline(ss, semestreStr, ',');
            getline(ss, gpaStr, ',');
            getline(ss, skillStr, ',');

            Estudiante estudiante;

            estudiante.student_id = stoi(idStr);
            estudiante.full_name = nombre;
            estudiante.career = carrera;
            estudiante.semester = stoi(semestreStr);
            estudiante.gpa = stof(gpaStr);
            estudiante.skill_score = stoi(skillStr);

            insertar(estudiante);
        }

        archivo.close();

        cout << "CSV CARGADO CORRECTAMENTE.\n";
    }
};

int main() {

```

```

cout << "=====\n";
cout << "Nombre: Emily Monterroso\n";
cout << "Carne: 9941-24-1489\n";
cout << "=====\n";

HashTable hash;

hash.cargarCSV("../estudiantes.csv");

int opcion = 0;

do {

    cout << "\n===== MENU =====\n";
    cout << "1. Mostrar tabla hash\n";
    cout << "2. Buscar estudiante\n";
    cout << "3. Eliminar estudiante\n";
    cout << "4. Mostrar estadísticas\n";
    cout << "5. Salir\n";
    cout << "Seleccione una opcion: ";

    cin >> opcion;

    if (opcion == 1) {

        hash.mostrarTabla();
    }

    else if (opcion == 2) {

        int idBuscar;

        cout << "Ingrese ID a buscar: ";
        cin >> idBuscar;

        Estudiante* encontrado = hash.buscar(idBuscar);

        if (encontrado != nullptr) {

            cout << "\n=== ESTUDIANTE ENCONTRADO ===\n";

            cout << "ID: "
                 << encontrado->student_id << endl;

            cout << "Nombre: "
                 << encontrado->full_name << endl;

            cout << "Carrera: "
                 << encontrado->career << endl;

        } else {

            cout << "No encontrado.\n";
        }
    }

    else if (opcion == 3) {

        int idEliminar;
    }
}

```

```
        cout << "Ingrese ID a eliminar: ";
        cin >> idEliminar;

        if (hash.eliminar(idEliminar)) {

            cout << "Eliminado correctamente.\n";

        } else {

            cout << "No existe estudiante.\n";

        }

    }

    else if (opcion == 4) {

        hash.mostrarEstadisticas();

    }

    else if (opcion == 5) {

        cout << "Saliendo...\n";

    }

    else {

        cout << "Opcion invalida.\n";

    }

} while (opcion != 5);

return 0;
```

EJECUCION


```
Project: tarea_hash_csvv
Run: tarea_hash_csvv
C:\Users\monte\OneDrive\Desktop\tarea-hash_csvv\cmake-build-debug\tarea_hash_csvv.exe
=====
Nombre: Emily Monterroso
Carne: 9941-24-1489
=====
CSV CARGADO CORRECTAMENTE.

===== MENU =====
1. Mostrar tabla hash
2. Buscar estudiante
3. Eliminar estudiante
4. Mostrar estadísticas
5. Salir
Seleccione una opcion:2

Ingrese ID a buscar:1001

=== ESTUDIANTE ENCONTRADO ===
ID: 1001
Nombre: Juan Perez
Carrera: Ingenieria en Sistemas

===== MENU =====
1. Mostrar tabla hash
2. Buscar estudiante
3. Eliminar estudiante
4. Mostrar estadísticas
5. Salir
```

```
Project: tarea_hash_csvv
Run: tarea_hash_csvv
Seleccione una opcion:1

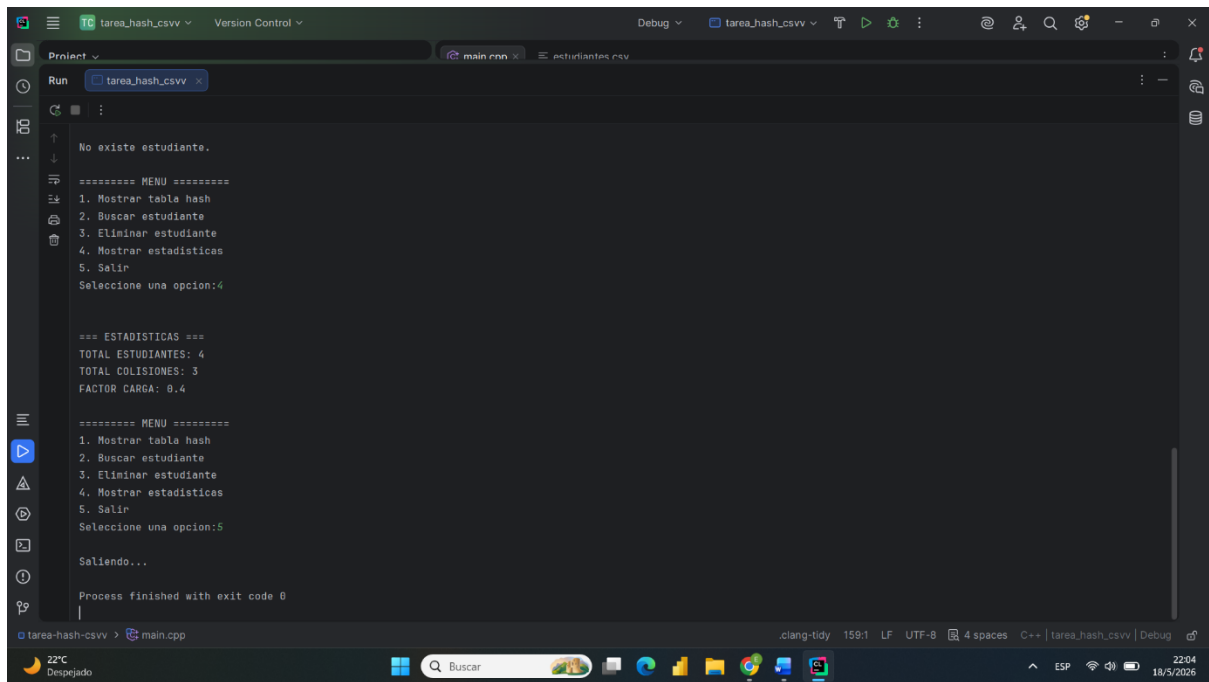
=== TABLA HASH ===
Bucket [0]: FINAL
Bucket [1]: (1031, Maria Gomez) -> (1021, Carlos Ruiz) -> (1011, Ana Lopez) -> (1001, Juan Perez) -> FINAL
Bucket [2]: FINAL
Bucket [3]: FINAL
Bucket [4]: FINAL
Bucket [5]: FINAL
Bucket [6]: FINAL
Bucket [7]: FINAL
Bucket [8]: FINAL
Bucket [9]: FINAL

===== MENU =====
1. Mostrar tabla hash
2. Buscar estudiante
3. Eliminar estudiante
4. Mostrar estadísticas
5. Salir
Seleccione una opcion:3

Ingrese ID a eliminar:1003

No existe estudiante.

===== MENU =====
1. Mostrar tabla hash
```



```
Run tarea_hash_csvv x
No existe estudiante.

===== MENU =====
1. Mostrar tabla hash
2. Buscar estudiante
3. Eliminar estudiante
4. Mostrar estadísticas
5. Salir
Seleccione una opcion:4

=== ESTADISTICAS ===
TOTAL ESTUDIANTES: 4
TOTAL COLISIONES: 3
FACTOR CARGA: 0.4

===== MENU =====
1. Mostrar tabla hash
2. Buscar estudiante
3. Eliminar estudiante
4. Mostrar estadísticas
5. Salir
Seleccione una opcion:5

Saliendo...

Process finished with exit code 0
```

La resolución de colisiones se implementó mediante

Separate Chaining utilizando listas enlazadas.

Esto puede visualizarse en el Bucket [1], donde

varios estudiantes comparten el mismo bucket

y se enlazan mediante nodos dinámicos.

Se realiza el bucket enlazando nodos

```
Bucket [1]: (1031, Maria Gomez) -> (1021, Carlos Ruiz) -> (1011, Ana Lopez) -> (1001, Juan Perez) -> FINAL
```